

COMP101: Introduction to Computing & Python

Final Exam Review Sheet



University Mohammed VI Polytechnic
School of Applied Science and Engineering (SASE)



Lecture-by-lecture guide with answer cues and mixed final-exam style prompts

How to use this document

- This is a **review sheet**, not a substitute for studying your lecture. Each section compresses one part of the semester into exam-ready reminders.
- Read each lecture in this order: **core ideas** → **pitfalls** → **answer cues** → **possible final questions**.
- Question tags: **[TA]** text answer, **[MCQ]** single-choice, **[MS]** multi-select, **[FIB]** fill in the blank, **[TF]** token fill, **[CODE]** short coding / tracing.
- When a cue is given, it is not the full answer. It points to the idea the examiner expects you to mobilize.

Global exam checklist

Before the final, make sure you can do all of the following without guessing:

- Trace assignments, conditionals, loops, and function calls line by line.
- Explain why one data structure is more natural than another.
- Distinguish syntax errors, runtime errors, and logic errors.
- Read a short code fragment and predict its output exactly.
- Write a small correct Python function from a plain-English specification.
- Explain the relevant system layer when something goes wrong: code, runtime, OS, or hardware.

Lecture 1 — Intro to Computing

Core ideas to review

- A computer is a very fast but very literal machine that stores data and executes instructions.
- Computer science is not only about programming; it studies representation, algorithms, systems, limits of computation, and ways of solving problems precisely.
- A program is a concrete implementation; an algorithm is a step-by-step method independent of one programming language.
- Keep four words separate: **data**, **information**, **program**, and **algorithm**.
- Distinguish **unsolvable** problems (no algorithm can solve all cases) from **intractable** problems (solvable in principle, but too expensive in practice).
- The input-processing-output-storage view is a useful first mental model for many systems.
- Correctness and efficiency are different questions: a slow correct algorithm is still correct; a fast wrong algorithm is still wrong.
- Von Neumann architecture: memory stores both instructions and data; CPU executes instructions; input/output devices communicate with the outside world.

Common traps

- Saying “computer science = coding” is too narrow.
- Saying “hard” when the real concept is “unsolvable” or “intractable.”
- Treating an algorithm and a program as synonyms.

Answer cues

- If asked “What is an algorithm?”, mention **finite**, **ordered steps**, **unambiguous instructions**, and a **goal**.
- If asked to compare unsolvable and intractable, say **impossible in principle** versus **possible but not realistically affordable**.

- If asked about Von Neumann architecture, mention **CPU, memory, input/output, stored-program idea**.

Possible final questions

- (1) [TA] Explain the difference between a program and an algorithm in two or three sentences. *Cue: one is language-independent, one is an implementation.*
- (2) [MCQ] Which statement best describes an intractable problem?
A. No algorithm exists. B. A solution exists but may be too expensive to compute. C. It cannot be described precisely. D. It can only be solved with Python.
- (3) [MS] Select all components that belong to the Von Neumann picture of a computer: CPU, memory, browser tab, input/output devices, file extension.
- (4) [FIB] A computer executes instructions very fast, but it is also very _____.

Lecture 2 — Intro to Python: values, variables, and basic I/O

Core ideas to review

- REPL = Read, Eval, Print, Loop. It is useful for fast experiments, but scripts are better for reusable programs.
- Distinguish values, literals, variables, and types.
- Assignment binds a name to a value; it is not a mathematical equation.
- Core built-in types: `int`, `float`, `bool`, `str`.
- Know arithmetic operators, comparison operators, bitwise operators, and precedence at a practical level.
- `input()` returns a string. Convert it explicitly when a number is needed.
- `print()` can control separators and line endings through `sep=` and `end=`.
- f-strings mix text and computed values clearly.
- ASCII basics: `ord` turns a character into a code point, `chr` goes back from code to character.
- Conversions matter: decimal to binary thinking, string to int, int to float, truth values in expressions.

Common traps

- Forgetting that `input()` returns `str`.
- Confusing `=` with `==`.
- Forgetting quotes around string literals.
- Mixing string concatenation with numeric addition carelessly.
- Using bitwise operators `&` `|` `^` when the question is actually about logical operators `and/or/not`.

Answer cues

- When asked what the REPL is for, say **interactive testing and immediate feedback**.
- When asked to explain `print(sep=..., end=...)`, mention **separator between arguments and what is printed after the last argument**.
- If a line combines user input with arithmetic, first ask: **do I need conversion?**

Possible final questions

- (1) [MCQ] What is the type of the result of `input()`?
A. `int` B. `str` C. `float` D. depends on what the user types
- (2) [TF] Complete the code so it prints A-B-C on one line:
`print("A", "B", "C", _____)`
- (3) [TA] Give one difference between running code in the REPL and running a Python script file.
- (4) [TA] Convert 15654 to binary.
- (5) [TA] convert 15468 to base 13.

(6) [CODE] What is printed?

```
name = "Sara"
age = 19
print(f"{name} is {age} years old")
print(ord("A"), chr(66))
```

Cue: know exact spacing and the meaning of ord/chr.

Lecture 3 — Control structures

Core ideas to review

- Branching chooses among alternatives: `if`, `elif`, `else`.
- Loops repeat work: `for` is natural when iterating over a range or collection; `while` is natural when repetition depends on a condition.
- Know comparison operators and logical combinations.
- `range(start, stop, step)` is half-open: it includes `start` and excludes `stop`.
- `enumerate` is useful when you need both index and value.
- `break` exits the loop immediately; `continue` skips the rest of the current iteration.
- Students should be able to trace nested conditionals and nested loops line by line.
- Off-by-one errors are one of the most common sources of wrong answers.

Common traps

- Using `while` without changing the state that controls termination.
- Misreading `range(5)` as producing 1 through 5.
- Putting `break` or `continue` in the wrong branch.
- Confusing chained comparisons and separate comparisons.

Answer cues

- If asked “for or while?”, answer by focusing on **what controls the repetition**.
- If asked to debug a loop, check **initialization, condition, update**. That trio catches many mistakes.
- If asked about `range`, explicitly mention the **excluded upper bound**.

Possible final questions

- (1) [FIB] The expression `range(2, 8, 2)` produces: _____
- (2) [MS] Select all true statements:
- `break` skips one iteration and continues.
 - `continue` skips the rest of the current iteration.
 - A `while` loop can become infinite if the condition never becomes false.
 - `range(4)` contains 4.
- (3) [TA] Explain one situation where `while` is more natural than `for`. *Cue: repetition until a condition changes.*
- (4) [CODE] Write a short loop that prints all even numbers from 0 to 10 inclusive.

Lecture 4 — Functions

Core ideas to review

- Functions package reusable logic, reduce duplication, and improve readability/testing.
- Distinguish **definition** from **call**, and **parameter** from **argument**.
- `return` sends a value back to the caller; `print` only displays something.
- `pass` is a placeholder that does nothing.
- Local variables live inside the function scope.

- Good function design often means one clear task, clear inputs, and a predictable output.
- Decomposition matters: split a large task into smaller helper functions.

Common traps

- Printing the result when the question asked to return it.
- Forgetting to return anything, which means Python returns `None`.
- Using a variable from outside the function when it should have been passed as an argument.

Answer cues

- If asked why functions matter, mention **reuse, decomposition, readability, testing**.
- If asked about parameters versus arguments, say **parameters belong to the function definition; arguments are the values passed at call time**.
- If a function computes something, ask: **should that value be returned?**

Possible final questions

- (1) [MCQ] What does a function return if execution reaches the end with no `return` statement?
A. 0 B. empty string C. `None` D. last printed value
- (2) [TA] Explain the difference between `return x` and `print(x)`.
- (3) [TF] Complete the function header: `def area_____`:
- (4) [CODE] Write a function `is_even(n)` that returns `True` if `n` is even and `False` otherwise.

Lecture 5 — Strings and lists

Core ideas to review

- Strings are sequences of characters; lists are sequences of arbitrary Python values.
- Know indexing, slicing, negative indices, and `len`.
- Strings are immutable; lists are mutable.
- Common string methods: `lower`, `strip`, `split`, `replace`, `find`.
- You should be able to iterate over strings and lists with `for`.
- For lists, know the basic update operations such as indexing assignment and methods like `append` or `pop`.
- Slicing creates sub-sequences and is often useful for copying or extracting parts.

Common traps

- Thinking that changing one character in a string by assignment is allowed.
- Confusing `s[1:4]` with `s[1]`.
- Forgetting that `split()` usually returns a list of strings.
- Forgetting the end index is excluded in slicing.

Answer cues

- If asked whether to use a string or a list, ask whether you need a **mutable sequence**.
- If asked what `strip()` does, say it removes surrounding whitespace, not internal characters in the middle.
- If asked to explain slicing, say **start included, stop excluded**.

Possible final questions

- (1) [FIB] If `s = "python"`, then `s[1:4]` is _____.
- (2) [MCQ] Which statement is true?
A. Strings are mutable. B. Lists are immutable. C. Strings are immutable. D. Neither strings nor lists

can be sliced.

(3) [MS] Which methods are string methods? `split`, `append`, `strip`, `lower`

(4) [CODE] Write code that reads a sentence and prints the number of words by using `split()`.

Lecture 6 — Dictionaries and sets

Core ideas to review

- A dictionary maps keys to values; a set stores distinct elements.
- Dictionaries are natural for lookup tables, counting, grouping, and frequency maps.
- Sets are natural for membership testing and deduplication.
- Know the difference between `d[key]` and `d.get(key)` at a basic level.
- Know how to iterate through dictionary keys, values, and items.
- Frequency counting is a major pattern for finals: scan the data, update counts in a dictionary.
- Be able to decide whether duplicates matter and whether order matters.

Common traps

- Using a set when the problem needs counts.
- Using a list for repeated membership tests when a set is the more natural structure.
- Forgetting that dictionary keys must be hashable.
- Confusing keys and values during iteration.

Answer cues

- If the question says **count**, think **dictionary**.
- If the question says **distinct**, **already seen**, or **membership**, think **set**.
- If asked why a set is insufficient for frequency counting, say it stores **presence**, not **multiplicity**.

Possible final questions

- (1) [TA] When is a set more natural than a list? Give one concrete example.
- (2) [MCQ] Which structure is the most natural for mapping student ID to grade?
A. list B. set C. dict D. string
- (3) [TF] Complete the update for a frequency count:
`freq[x] = freq.get(x, 0) + _____`
- (4) [CODE] Write a short function that takes a list of integers and returns a dictionary counting how many times each integer appears.

Lecture 7 — Problem solving patterns

Core ideas to review

- Pattern recognition, decomposition, abstraction, and algorithm design are not slogans; they are ways to attack unfamiliar problems.
- Common exam patterns: linear scan, running sum, filtering, mapping, counting, min/max tracking, preprocessing.
- `map`, `filter`, `scan`, and `accumulate` are not separate worlds; they are recurring transformations on collections.
- Static range sum queries motivate prefix sums: do extra work once so each later query becomes cheap.
- “How many zeros in $n!$?” is solved by counting factors of 5, not by computing the gigantic factorial itself.
- In many questions, the real challenge is representing the state that must be maintained while scanning.

Common traps

- Jumping into code before identifying the pattern.
- Recomputing the same information again and again instead of preprocessing.
- Solving the wrong problem because the state to maintain was not identified clearly.

Answer cues

- Ask: **What must be remembered from left to right?**
- Ask: **Can I preprocess once and answer many queries fast?**
- For trailing zeros in $n!$, mention **pairs of 2 and 5, where 5 is the limiting factor.**

Possible final questions

- (1) [TA] Why is computing $n!$ directly a poor strategy for counting the number of trailing zeros? *Cue: count factors, not the full value.*
- (2) [MS] Select all problems where prefix sums are a natural idea:
 - many queries asking for sums on subarrays
 - checking whether a number is even
 - repeated range-sum questions on an unchanged array
 - counting words in one short sentence only once
- (3) [FIB] In a left-to-right scan that tracks the maximum seen so far, the maintained state is _____.
- (4) [CODE] Given a list `a`, write code to build a prefix-sum list `pref` where `pref[i]` stores the sum of the first `i` elements.

Lecture 8 — Sorting and searching

Core ideas to review

- Understand sorting algorithms we saw in class (Bubble sort, Selection sort, Insertion sort).
- Linear search checks elements one by one and works on any list.
- Binary search needs sorted data and repeatedly halves the search interval.
- Be able to explain the idea of binary search in words, not only write code.
- The hardest part of binary search is handling boundaries correctly.
- Sorting is often used as preprocessing to make later operations easier.
- On an exam, if the data is not sorted, binary search is usually not directly applicable.

Common traps

- Applying binary search to an unsorted list.
- Updating the wrong half after comparing with the middle element.
- Off-by-one boundary mistakes.

Answer cues

- If asked why binary search is fast, say it **discards half of the remaining candidates each step.**
- If asked whether binary search can be used, first state the condition: **the search space must be ordered in a usable way.**
- If tracing the algorithm, always track `left`, `right`, `mid`.

Possible final questions

- (1) [MCQ] Which condition is required for standard binary search on a list?
A. all elements distinct B. list sorted C. list contains only integers D. list length is even

- (2) [TA] Explain the main idea of binary search in plain English.
- (3) [TF] Complete the middle-index formula: `mid = (left + right) // _____`
- (4) [TA] Write the steps of bubble sort on a given list.
- (5) [CODE] Write a function that returns `True` if a target is present in a sorted list and `False` otherwise, using binary search.

Lecture 9 — Recursion and recurrence

Core ideas to review

- Recursion solves a problem by reducing it to smaller instances of the same problem.
- Every correct recursive solution needs a base case and a recursive case.
- You must be able to trace calls using the call stack idea.
- Canonical examples: factorial, Fibonacci, gcd, Towers of Hanoi.
- Recurrence relations describe how values or work depend on smaller subproblems.
- The tiling problem for a $1 \times n$ row with 1×1 and 2×1 tiles leads naturally to a Fibonacci-like recurrence.

Common traps

- Missing or wrong base case.
- Recursive call that does not move toward termination.
- Treating the recursive formula and the code as if they were the same thing without checking the actual stopping conditions.

Answer cues

- If asked to justify recursion, explicitly say **smaller instance of the same structure**.
- For gcd, mention the Euclidean idea: **replace** (a, b) **with** $(b, a \% b)$.
- For tiling, ask what the first tile could be; this usually reveals the recurrence.

Possible final questions

- (1) [FIB] A recursive algorithm must have at least one _____ case.
- (2) [TA] Explain why the recurrence for the tiling problem resembles Fibonacci. *Cue: split by the shape placed first.*
- (3) [MS] Which of the following are standard examples of recursion from the course? factorial, Fibonacci, gcd, binary search, Hanoi
- (4) [CODE] Write a recursive function `factorial(n)` for nonnegative integers.

Lecture 10 — Working with files

Core ideas to review

- Files allow programs to persist data beyond one execution.
- Know `open()`, `read()`, `write()`, and the difference between reading the whole file versus processing line by line.
- Understand file modes at a basic level: read, write, append.
- A context manager with `with open(...)` ensures cleanup even if an error happens.
- Know why closing a file matters.
- Many practical tasks combine file reading with string processing or counting patterns.

Common traps

- Opening a file in write mode when you intended to read it.
- Forgetting that writing may overwrite existing content in some modes.
- Forgetting to close files when not using a context manager.

Answer cues

- If asked why with `open(...)` is good style, say it **automatically closes the file and handles cleanup reliably**.
- If asked about append versus write, say append adds at the end, while write mode may replace previous content.

Possible final questions

- (1) [MCQ] Which form is the safest standard way to open a file for reading text?
A. with `open("data.txt", "r") as f:` B. `open("data.txt") = f` C. `file("data.txt")`
D. `read("data.txt")`
- (2) [TA] What problem does the context-manager pattern solve for file handling?
- (3) [TF] Complete: with `open("notes.txt", "r") as f:`
`text = f._____()`
- (4) [CODE] Write code that counts how many lines are in a text file.

Lecture 11 — Object-oriented programming basics

Core ideas to review

- OOP groups data and behavior together through objects.
- A class describes a blueprint; an object is an instance.
- Attributes store object state; methods define behavior.
- `self` refers to the current instance inside instance methods.
- Encapsulation means hiding or controlling internal details behind an interface.
- Abstraction means focusing on the relevant behavior without exposing every implementation detail.
- Inheritance allows a class to reuse or extend another; polymorphism means different objects can respond to the same operation in their own way.
- Double underscore names trigger name mangling; they are mainly about avoiding accidental clashes, not absolute secrecy.

Common traps

- Confusing a class with an object.
- Forgetting `self` in method definitions.
- Thinking name mangling makes an attribute truly inaccessible.
- Using inheritance when simple composition or plain functions would be clearer.

Answer cues

- If asked why OOP exists, mention **organizing state and behavior together when that improves clarity**.
- If asked about encapsulation, say **the object exposes a controlled interface and protects internal representation choices**.
- If asked about polymorphism, say **same method name or interface, different behavior depending on the object**.

Possible final questions

- (1) [FIB] A class is a _____; an object is an _____ of that class.
- (2) [TA] Explain the difference between encapsulation and abstraction.
- (3) [MCQ] What is the role of `self` in an instance method?
A. It stores the class name. B. It refers to the current object. C. It is optional syntax. D. It creates inheritance.
- (4) [CODE] Define a class `Counter` with an attribute value initialized to 0 and a method `inc()` that increases it by 1.

Lecture 12 — Computer systems and execution layers

Core ideas to review

- A running Python program sits on top of several layers: hardware, operating system, Python runtime/interpreter, libraries, and your code.
- A **program** is passive code stored somewhere; a **process** is a running instance with its own execution state.
- User space and kernel space separate ordinary application code from privileged OS operations.
- The stack is associated with function-call frames and local execution state; the heap is associated with dynamically managed objects.
- Debugging improves when you ask which layer is responsible for the failure.

Common traps

- Treating “Python” as if it were the only layer that exists.
- Confusing program and process.
- Thinking stack and heap are just synonyms for fast and slow memory.

Answer cues

- If the question asks where to look first, classify the failure: **syntax? logic? missing file? OS permission? crash outside your code?**
- If asked stack versus heap, keep it simple: **call-related execution frames versus dynamically managed objects.**
- If asked program versus process, say **stored code versus code currently executing.**

Possible final questions

- (1) [TA] Explain the difference between a program and a process.
- (2) [MS] Select all layers that may be involved when Python code reads a file: hardware, operating system, Python runtime, your code, the moon.
- (3) [MCQ] Which statement about kernel space is the most accurate?
A. It is where every Python variable is stored. B. It is the privileged part of the OS. C. It is another word for REPL. D. It is the same as the heap.
- (4) [FIB] When a function is called, a new _____ frame is created.

Lecture 13 — Computational thinking and strategic problem solving

Core ideas to review

- Pattern recognition: compare the new problem to familiar structures.
- Decomposition: split a large vague task into smaller subproblems.
- Abstraction: keep the essential features and ignore irrelevant noise.
- Algorithm design: turn the chosen idea into precise steps.
- Testing with small examples and edge cases is part of thinking, not a last-minute repair step.
- Simple strategy games such as Nim highlight the importance of invariants, patterns, and reasoning before coding.

Common traps

- Starting with syntax instead of understanding the structure of the problem.
- Failing to test edge cases such as empty input, one-element input, or already-solved cases.
- Confusing abstraction with leaving out essential information.

Answer cues

- If asked which computational-thinking pillar is being used, justify it with the action performed: **breaking apart**, **spotting similarity**, **simplifying**, or **specifying steps**.
- In Nim-style reasoning questions, look for a **state pattern or invariant**, not random move descriptions.

Possible final questions

- (1) [TA] Give one example of decomposition for a program that reads a file and counts word frequencies.
- (2) [MCQ] Which computational-thinking idea is most directly involved when you ignore irrelevant details and keep only the structure that matters?
A. decomposition B. abstraction C. looping D. inheritance
- (3) [MS] Which actions are good exam habits? test a tiny example, consider edge cases, guess output without tracing, choose a state to maintain.
- (4) [FIB] Before coding, a useful question is: “What information must be _____ from one step to the next?”

Cross-topic synthesis: problems that combine several lectures

- A file-processing question may combine strings, loops, functions, dictionaries, and debugging.
- A binary-search question may also check whether you noticed the precondition “sorted.”
- A recursion question may also test your understanding of return values and base cases.
- An OOP question may still be a simple function-design question in disguise: identify state, identify behavior, decide the interface.
- Many “hard” questions become manageable once you identify the correct representation and the correct pattern.

Integrated mixed practice for the final

- (1) [TA] A program reads a text file, normalizes each word with `lower()` and `strip()`, then counts frequencies. Name at least four course topics involved.
- (2) [MS] Select all good first questions to ask before coding a solution:
 - What state must be maintained?
 - Does order matter?
 - Do I need counts or just distinct values?
 - Can I apply binary search even though the data is unsorted?
- (3) [TF] Complete the Euclidean-step idea for gcd: replace (a, b) by $(b, \text{_____})$.
- (4) [CODE] Write a function that takes a filename, reads all lines, and returns the number of non-empty stripped lines.

Last-minute checklist before the exam

- Can I explain each major concept in one or two clean sentences?
- Can I predict Python output exactly, including spaces and line breaks?
- Can I write a short function with the correct return value?
- Can I justify my choice of list, set, or dictionary?
- Can I recognize when a problem is about scanning, counting, preprocessing, recursion, or search?
- Can I tell whether an error is syntactic, runtime, or logical?